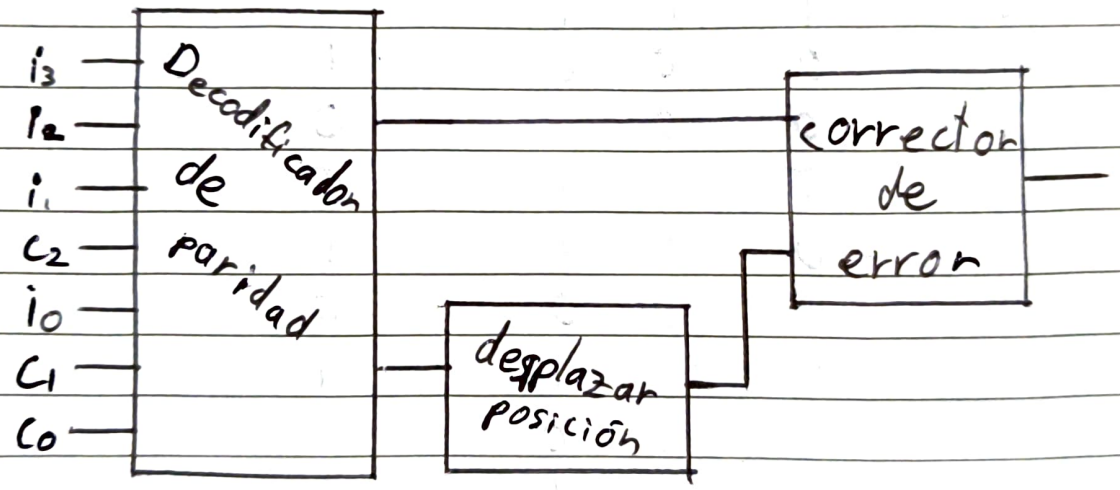
Proyecto 1
Diseño Lógico
Grupo 1
I Semestre 2026

Integrantes:
- David Araya Mora
- Nicolás Castro
- Sebastián Fallas Mora

• Primer módulo: Decodificador de paridad



Se utilizó un XOR para calcular la paridad correspondiente a cada bit de seguridad y descubrir la posición del error.

¡Marca un norte en tu camino!

2

Ecuación booleana correspondiente:

- Primer XOR: síndrome [0]

c_0	i_0	i_1	i_3	síndrome [0]
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

¡Marca un norte en tu camino!

- Segunda XOR: síndrome [1]

c_1	i_0	i_2	i_3	síndrome [1]
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

¡Marca un norte en tu camino!

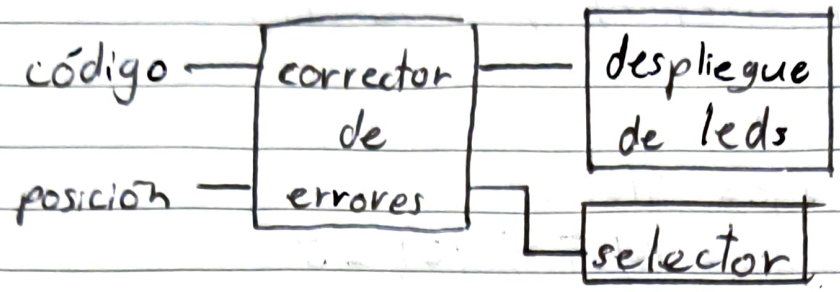
- Tercer XOR: síndrome [2]

c_2	i_1	i_2	i_3	síndrome [2]
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Ya teniendo los 3 bits que indican ~~los~~ la posición, se pasa ~~a un "subsistemódulo"~~ por el desplazador de posición ya que se quiere que la primera posición sea 000 en vez de 001 (lo que da este método) y que no tener errores sea 111 y no 000; se guarda en "posicion"

¡Marca un norte en tu camino!

- Segundo módulo: Corrector de errores



Recibiendo código y posición, se puede arreglar el código recibido y enviar una nueva ~~(cab)~~ variable llamada ~~(código)~~ "codigo_corregido" a los siguientes módulos; las ecuaciones correspondientes al proceso:

~~(cód)~~

$$codigo_corregido[0] = codigo[0] \oplus (\bar{p}_2\ \bar{p}_1\ \bar{p}_0)$$
$$codigo_corregido[1] = codigo[1] \oplus (\bar{p}_2\ \bar{p}_1\ p_0)$$
$$codigo_corregido[2] = codigo[2] \oplus (\bar{p}_2\ p_1\ \bar{p}_0)$$
$$codigo_corregido[3] = codigo[3] \oplus (\bar{p}_2\ p_1\ p_0)$$
$$codigo_corregido[4] = codigo[4] \oplus (p_2\ \bar{p}_1\ \bar{p}_0)$$
$$codigo_corregido[5] = codigo[5] \oplus (p_2\ \bar{p}_1\ p_0)$$
$$codigo_corregido[6] = codigo[6] \oplus (p_2\ p_1\ \bar{p}_0)$$

- Simulaciones para primeros módulos

  se creó un testbench instando el ~~segundo mod~~ primer módulo. ~~Es~~ Este TB utiliza un código determinado en este mismo, lo manda al módulo decodificador

¡Marca un norte en tu camino!

y devuelve luego el código ingresado (todavía sin arreglar) y la posición:

- Simulación 1:

  código = 1110101

  posición = ~~(6) 5 =)~~ $101_2 = 5_{10}$

- Simulación 2:

  código = 1010101

  posición = 111_2 → no hay error

- Simulación 3:

  código = 1111001

  posición = $000_2 = 0_{10}$

Viendo que el decodificador funciona, se procedió a crear un TB para ~~prop~~ probar el decodificador en conjunto con el corrector, en este se imprime el código, la posición y el código corregido:

- Simulación 1:

  código = 1110101

  posición = 101

  código corregido = 1010101.

¡Marca un norte en tu camino!

- Simulación 2:

  código = 1010101

  posición = 111

  código corregido = 1010101

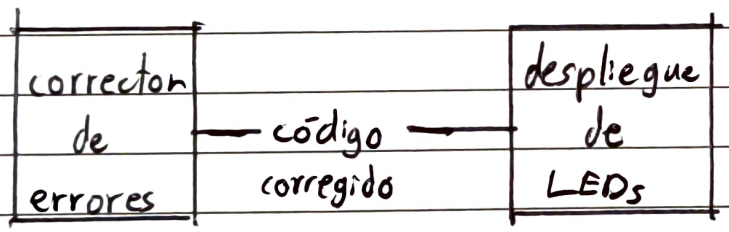
- Simulación 3:

  código = ~~(1010101)~~ 1111001

  posición = 0

  cc = 1111000

• Tercer módulo: Despliegue de LEDs



Se toma de "código_corregido" los bits en la posición 6, 5, 4 y 2, luego se asigna a los primeros 4 LEDs de la FPGA.

• Simulación agregando tercer módulo

Se usaron las mismas tres códigos de prueba pero teniendo el módulo 3 implementado y los constraints agregados, mostrando los códigos:

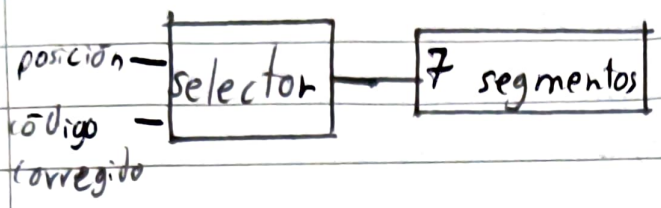
Sim 1: 1011

Sim 2: 1011

Sim 3: 1110

¡Marca un norte en tu camino!

- Quinto módulo: 7 segmentos



¡Marca un norte en tu camino!